Angular Assignment

Date of upload: 8th July 2026
Date of Correction: 8th July 2026
Date of submission: 19 July 2026

---

1. **Overview:** The assignment is based on a partially developed **Habit Management System**. The completed portion of the project will be provided as both an illustrative example and the starting point for the assignment. You will receive the existing source code together with a brief explanation of its structure and implementation. The remaining features of the Habit Management System must then be completed as part of the assignment. The lecture videos released subsequently will examine the provided implementation in greater detail and explain the Angular concepts used in developing it.

2. **Description:** The Habit Management System is organised as a collection of Angular components. Each component is responsible for one part of the user interface and its associated behaviour. Navigation between the major components is handled through Angular routing.

   The primary components are:

   1. **App component:** The root component of the Angular application. It provides the main application shell and displays the component selected by the router.
   2. **Authentication component:** Allows a user to register or log in. It communicates with the backend authentication service and stores the authentication information returned after a successful login.
   3. **Habit component:** Allows users to create, view, update and delete habits, and to record whether a habit has been completed.
   4. **Streak Component:** This presents progress information derived from habit-completion records, such as the number of consecutive days for which a habit has been completed.

   The Angular frontend communicates with a **`FastAPI`** backend. The backend performs authentication, stores application data in **`SQLite`** databases, and exposes **`HTTP`** endpoints used by the Angular services.

3. **Starting the Angular Application:** Let us describe in some detail how an Angular application starts. When we run:

   ```
   ng serve --open
   ```

   the Angular build system begins processing **`main.ts`** and the files on which it transitively depends, including **`TypeScript`** files, **`HTML`** templates, stylesheets, and external dependencies. It also starts a development server.

   When a browser visits **`http://localhost:4200/`**, it sends a request to the development server. The server responds with the application page based on **`index.html`**. The browser

parses this page, requests the generated JavaScript resources, and executes the JavaScript **main.js** compiled from **main.ts**. **main.ts** contains a call of the form:

```
bootstrapApplication(App, appConfig);
```

This call tells Angular to bootstrap **App** as the root component. Angular reads the metadata of **App**, its selector identifies **<app-root>** in **index.html** as the host element, and its associated template is rendered inside that element.

The process then continues recursively. Suppose the template of **App** (in its metadata or a separate html file) contains:

```
<app-x></app-x>
```

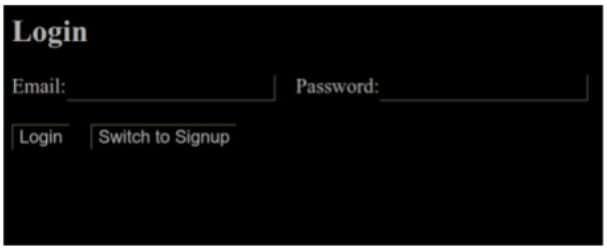
Angular finds the component available to **App** whose selector is **app-x**, creates that component, and renders its template inside **<app-x></app-x>**. In this way, the component tree grows from the root component. One important exception is:

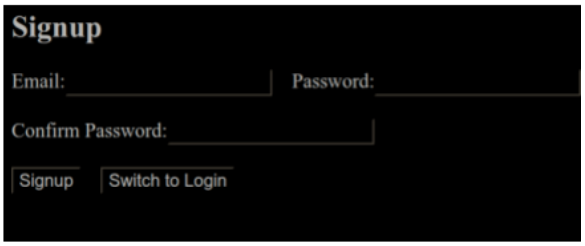
```
<router-outlet></router-outlet>
```

Here, the component inserted into the outlet is selected by the Angular router according to the current URL, rather than directly through a component selector appearing in the template.

The relationship between URLs and components is defined in **app.routes.ts**. This file contains the application's routing table. Each route associates a path, such as **auth**, **habits** or **streak**, with the component that should be displayed. It may also contain redirects, such as a default route that sends the user to the authentication page.

1. **The Authentication Component (auth.component.ts and auth.component.html):**





The authentication component provides both **login** and **signup** facilities through a single Angular component **auth.component.ts**. Instead of maintaining two separate pages, it stores the current operating mode in the component class.

Initially, the component operates in **login** mode. The user may switch between the two modes using the button provided in the template.

The Template: auth.component.html: The outer ** ** and **<form>** provide the visual structure of the authentication page. The form uses Angular's **ngSubmit** event:

```
<form (ngSubmit)="onSubmit()">
```

When the user submits the form, Angular calls the **onSubmit()** method defined in **auth.component.ts**.

The headings and input fields are displayed conditionally using ***ngIf**. For example:

```
<h2 *ngIf="mode === 'login'">Login</h2>
<h2 *ngIf="mode === 'signup'">Signup</h2>
```

The same idea is used to display either the **login** fields or the **signup** fields.

The login form contains fields for an email address and a password. These fields use two-way

binding:

```
[(ngModel)]="loginData.email"
[(ngModel)]="loginData.password"
```

Two-way binding connects the input element to a property in the component class. When the user types into the input field, the corresponding property is updated. If the property is changed by the component, the input field is also updated. The signup fields are similarly connected to **signupData**.

The confirmation-password field is displayed only in signup mode. It allows the component to verify that the user entered the intended password correctly.

Each input contains a **name** attribute because the fields occur inside a template-driven Angular form. The **required** attribute prevents submission when a compulsory field is empty, while **type="email"** and **type="password"** provide the corresponding browser input behaviour.

The label of the submit button is selected using a **?:** expression:

```
{{ mode === 'login' ? 'Login' : 'Signup' }}
```

The second button calls **toggleMode()**:

```
<button type="button" (click)="toggleMode()">
```

Its **type** is deliberately set to **button**; otherwise, a button placed inside a form may submit the form. The template also displays messages produced by the component. An error message is shown when **errorMessage** contains a value, while a success message is shown after an authentication response has been received.

The Component Class: AuthComponent

The **@Component** decorator declares **AuthComponent** as a standalone Angular component:

```
@Component({
  selector: 'app-auth',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './auth.html',
  styleUrl: './auth.css'
})
```

The selector **app-auth** identifies the component when it is used in another template. **CommonModule** supplies facilities such as ***ngIf**, while **FormsModule** supplies **ngModel** and template-driven form support.

The component maintains separate objects for **login** and **signup** data:

```
loginData = { email: '', password: '' };
signupData = { email: '', password: '', confirmPassword: ''};
```

Keeping these objects separate avoids mixing values entered in the two modes.

The constructor requests two Angular services:

```
constructor( private authService: AuthService, private router:
Router) {}
```

AuthService communicates with the backend authentication endpoints. **Router** is used to navigate to the dashboard after successful authentication. Angular's dependency-injection system supplies both objects when the component is created. **toggleMode()** method switches between login and signup:

The principal behaviour is implemented in **onSubmit()**. The method first examines **mode**. In login mode, it calls:

```
this.authService.login(this.loginData)
```

The service returns an **Observable**, representing an **HTTP** operation whose response will arrive later. The component subscribes to this observable and supplies two handlers:

```
subscribe({
  next: ...,
  error:...
});
```

The **next** handler runs when authentication succeeds. It stores the response, saves the returned token in browser storage, and navigates to the dashboard:

```
localStorage.setItem('token', response.token);
this.router.navigate(['/dashboard']);
```

The token can later be included in requests to protected backend endpoints, such as the habit-management endpoint.

The **error** handler runs when the backend rejects the login or when the request fails. It stores a user-facing message in **errorMessage**, causing the corresponding ** ** in the template to appear.

In **signup** mode, the component first checks whether the **password** and **confirmation password** agree. If they differ, the method records an error and returns without contacting the backend. Otherwise, it calls **authService.signup()** and handles the response in the same general manner as **login**.

Thus, the component combines several central Angular ideas: template directives and interpolation, two-way form binding, event handling, service-based **HTTP** communication, response handling with **Observable** subscription, token storage and router navigation.

The **HTML** template defines what the user sees, while the **TypeScript** class stores the state and controls the behaviour of the authentication page.

4. **The authentication backend:** The authentication backend is a small FastAPI service used by the Angular application. Angular sends an email and password; the backend stores or checks the user, hashes passwords with **bcrypt,** creates a JSON Web Token (**JWT**), and returns the token together with basic user information.

   Here is a summary of request and response exchange flow: Let us say that during the signup operation, the client sends:

```
{
  "email": "amit@gmail.com",
  "password": "something"
}
```

When the user submits the Angular authentication form, the frontend sends the entered email address and password to the FastAPI backend using either the **POST**

/api/auth/signup endpoint or the **POST /api/auth/login endpoint**. The request first reaches **main.py**, which registers the authentication routes and passes the request to the appropriate function in **auth/auth.py**. That function uses **database.py** to obtain a database session and **models.py** to work with the **User** table. During signup, the password is converted into a secure **bcrypt** hash before the user record is stored; during login, the supplied password is checked against the stored hash. If authentication succeeds, **jwt_tokens.py** creates a signed **JSON** Web Token containing the user's ID. The backend then returns an **AuthResponse** containing this token together with basic user information:

```
{
  "token": "<JWT>",
  "user": {
    "id": 1,
    "email": "student@example.com"
  }
}
```

Angular can store the token and use it to prove the user's identity in later requests.

The backend files: The high-level authentication flow is implemented by several files, each responsible for one stage of the request.

i. **main.py:** The process begins in **main.py**, which creates the **FastAPI** application and attaches the authentication router using the prefix **/api/auth**. The router itself is defined in **auth/auth.py**. Combining the router prefix with the route names declared there produces the complete endpoints **POST /api/auth/signup** and **POST /api/auth/login. main.py** also enables cross-origin requests from **http://localhost:4200**, allowing the Angular development server to communicate with the **FastAPI** backend.

ii. **schemas.py:** When a request reaches one of these endpoints, **FastAPI** converts the incoming **JSON** data into an **AuthRequest** object defined in **schemas.py**. This schema specifies that the request must contain an **email** and a **password**. The endpoint function in **auth/auth.py** then obtains a **SQLAlchemy Session** through **Depends(get_db)**.

iii. **database.py:** This provides the database engine, session creation, and the **get_db()** dependency.

iv. **models.py:** The database session is used together with the **User** model defined in **models.py**. This model describes the **users** table, whose main fields are **id, email**, and **hashed_password**. The **id** is the primary key, while the email address is unique and indexed. The plain password is never written to the database.

v. **auth/auth.py**: During signup, **auth/auth.py** hashes the supplied password using **bcrypt** and stores only the resulting hash. During login, it retrieves the user by email and asks **bcrypt** to compare the supplied password with the stored hash.

vi. **jwt_tokens.py:** After a successful signup or login, **auth/auth.py** calls **create_access_token()** from **jwt_tokens.py**. This function creates a signed **JWT** using the **HS256** algorithm. The token remains valid for 120 minutes and stores the authenticated user's **ID** in its **sub** claim. Later, protected endpoints can call **get_user_id()** from the same file. This function reads the bearer token from the request, verifies its signature and expiry time, extracts the user **ID**, and returns it. An invalid or expired token results in a **401 Unauthorized** response.

vii. **`schemas.py`:** Finally, the signup or login handler returns an **`AuthResponse`**, also defined in **`schemas.py`**. This response contains the **`JWT`** and a user dictionary containing the user's ID and email address. Angular can retain the token and include it in later requests, allowing the backend to identify the user without requiring the email and password to be sent again.

Running and testing: From the backend directory, the usual development command is:

```
uvicorn main:app --reload
```

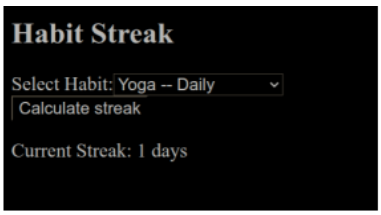
The server normally runs at **`http://localhost:8000`**.

5. **Extensions:** There are two extensions that you must implement. The Habit Component and the Streak component.

   a. **Habit Component:** The Habit component allows an authenticated user to create and manage daily or weekly habits. The form accepts a habit name and frequency and binds these values to the component's **`newHabit`** property. When the component loads, **`ngOnInit()`** requests the user's existing habits from the backend. **`HabitService`** attaches the **`JWT`** stored in **`localStorage`** to each request, allowing the backend to identify the current user and return only that user's habits. Submitting the form sends a **`POST`** request to **`/api/habit/`**, and the newly created habit is added to the displayed list. Clicking **`Complete`** sends the current date to **`/api/habit/{habitId}/complete`**, where the backend records completion for the appropriate daily or weekly period and rejects duplicate completions. Clicking **`Delete`** removes the selected habit from both the database and the displayed list.

   b. **Streak Component;** The Streak component allows an authenticated user to calculate the current streak for one of their habits. When the component loads, it uses **`HabitService`** to retrieve the user's daily and weekly habits. These habits are displayed in a drop-down list showing each habit's name and frequency. The **`Calculate`** streak button remains disabled until the user selects a habit. When clicked, the component requests all recorded completions for that habit from the backend. For a daily habit, it counts consecutive completed dates working backwards from today. If today has not yet been completed, the calculation begins with yesterday, so an existing streak is not broken prematurely. For a weekly habit, it similarly counts consecutive completed weeks using Monday as the beginning of each week. The calculation stops as soon as it encounters the first missing day or week. The interface then displays the resulting current streak or reports that no streak is available.

   Here are screenshots of Habit and Streak components:



   c. **The Overall Directory Structure:** Files in red color are the ones that were modified or introduced by me and have been passed to you. Files in blue are the ones that you should create. Feel free to create any other files that you may need.

```
angular/angular_app/
├── angular.json
├── node_modules/…
├── package.json
├── package-lock.json
├── public
│   └── favicon.ico
├── README.md
├── src
│   ├── app
│   │   ├── app.config.server.ts
│   │   ├── app.config.ts *
│   │   ├── app.css *
│   │   ├── app.html *
│   │   ├── app.routes.server.ts
│   │   ├── app.routes.ts *
│   │   ├── app.spec.ts
│   │   ├── app.ts *
│   │   ├── auth
│   │   │   ├── auth.css *
│   │   │   ├── auth.html *
│   │   │   ├── auth.service.ts *
│   │   │   ├── auth.spec.ts
│   │   │   └── auth.ts *
│   │   ├── habit
│   │   │   ├── habit.css **
│   │   │   ├── habit.html **
│   │   │   ├── habit.model.ts **
│   │   │   ├── habit.service.ts **
│   │   │   ├── habit.spec.ts
│   │   │   └── habit.ts **
│   │   └── streak
│   │       ├── streak.html **
│   │       └── streak.ts **
│   ├── backend
│   │   ├── app.db
│   │   ├── auth
│   │   │   ├── auth.db
│   │   │   └── auth.py *
│   │   ├── dashboard
│   │   ├── database.py *
│   │   ├── habit
│   │   │   └──habit.py**
│   │   ├── __init__.py
│   │   ├── jwt_tokens.py *
│   │   ├── main.py *
│   │   ├── models.py *
│   │   └── schemas.py *
│   ├── index.html
│   ├── main.server.ts
│   ├── main.ts
│   ├── server.ts
│   └── styles.css
├── tsconfig.app.json
├── tsconfig.json
└── tsconfig.spec.json
```